

บทที่ 6 การจำแนกประเภทข้อมูล

วัตถุประสงค์การเรียนรู้

1. อธิบายนิยาม หลักการ และกระบวนการของการจำแนกประเภทข้อมูล (Classification) ในการเรียนรู้แบบมีผู้สอน รวมถึงความแตกต่างระหว่าง Induction และ Deduction และปัญหา Overfitting
2. อธิบายทฤษฎี KNN รวมถึงการคำนวณ Euclidean Distance, Majority Vote, การปรับ weights และ trade-off ระหว่าง k กับ bias-variance
3. อธิบายทฤษฎี Decision Tree รวมถึงเกณฑ์ Gini Index, Entropy และ Information Gain และแนวทาง Pruning ด้วย Pre-pruning และ Post-pruning
4. อธิบายทฤษฎี SVM รวมถึง Maximum Margin Hyperplane, Support Vectors, Regularization Parameter C และ Kernel Trick (Linear, RBF, Polynomial)
5. อธิบายทฤษฎี Naive Bayes รวมถึง Bayes Theorem, สมมติฐาน Conditional Independence และ Gaussian Naive Bayes และทฤษฎี Logistic Regression รวมถึง Sigmoid Function, logit function และ Regularization (L1/L2)
6. อธิบายและคำนวณเมตริกการประเมินประสิทธิภาพโมเดลจาก Confusion Matrix ได้แก่ Accuracy, Precision, Recall และ F1-Score
7. สร้างและประเมินโมเดล KNN และ RNN ด้วย Scikit-learn สำหรับการจำแนกแบบ Binary และ Multiclass รวมถึงการปรับค่า k, radius และ weights พร้อม StandardScaler
8. สร้างและปรับแต่งโมเดล Decision Tree ด้วย criterion (Gini/Entropy) และ max_depth พร้อมแสดงผลโครงสร้างต้นไม้ด้วย tree.plot_tree() และ tree.export_text()

9. สร้างและประเมินโมเดล SVM, Naive Bayes (GaussianNB) และ Logistic Regression ด้วย Scikit-learn รวมถึงการปรับค่า C และเปรียบเทียบประสิทธิภาพของทุกอัลกอริทึมบนชุดข้อมูล Wine Dataset

6.1 บทนำ

การจำแนกประเภทข้อมูล (Classification) คือวิธีการเรียนรู้แบบมีผู้สอน (Supervised Learning) ที่ฝึก Algorithm เพื่อกำหนดป้ายกำกับที่กำหนดไว้ล่วงหน้าให้กับข้อมูลที่รับเข้ามา เป้าหมายคือการเรียนรู้ฟังก์ชันที่สามารถทำนายป้ายกำกับคลาสของข้อมูลที่ยังไม่เคยเห็นได้อย่างแม่นยำ ตัวอย่างปัญหาการจำแนกประเภทที่พบบ่อย ได้แก่ การรู้จำรูปภาพ การตรวจจับสแปม และการประมวลผลภาษาธรรมชาติ

วิธีการจำแนกประเภทที่นิยมใช้ใน Scikit-Learn มีดังนี้:

10. Logistic Regression: โมเดลเชิงเส้นที่ใช้สำหรับการจำแนกแบบ Binary โดยทำนายหนึ่งในสองคลาสที่เป็นไปได้
11. Decision Trees: โมเดลแบบต้นไม้ที่ใช้ชุดกฎ if-then ในการทำนาย
12. Random Forest: วิธี Ensemble ที่รวม Decision Tree หลายต้นเพื่อเพิ่มความแม่นยำ
13. Naive Bayes: โมเดลเชิงความน่าจะเป็นที่ทำนายตามความน่าจะเป็นของแต่ละคลาสที่กำหนด Feature ที่รับเข้ามา
14. Support Vector Machines (SVMs): โมเดลเชิงเส้นที่ค้นหาขอบเขตที่ดีที่สุดระหว่างคลาสโดยการเพิ่ม Margin ให้สูงสุด
15. K-Nearest Neighbors: วิธีที่ใช้ตัวอย่างที่มีป้ายกำกับ k รายการที่ใกล้เคียงที่สุดกับข้อมูลนำเข้าเพื่อทำนาย
16. Gradient Boosting: วิธี Ensemble ที่รวมโมเดลอ่อนแอหลายตัวเพื่อเพิ่มความแม่นยำ

6.2 แนวคิดและทฤษฎีที่เกี่ยวข้อง

6.2.1 นิยามและกระบวนการจำแนกประเภทข้อมูล

Tan, Steinbach และ Kumar (2006) ให้นิยาม Classification ว่าเป็นงานการเรียนรู้ target function f ที่แปลง attribute set X ของแต่ละ record ไปยัง class label y ที่กำหนดไว้ล่วงหน้า โดย target function นี้เรียกอีกชื่อว่า classification model ข้อมูลนำเข้าแต่ละ record ประกอบด้วย tuple (x, y) ซึ่ง x คือ attribute set และ y คือ class label ซึ่งต้องเป็น discrete attribute ลักษณะนี้แยก Classification ออกจาก Regression ที่ y เป็น continuous attribute สิ่งสำคัญคือ classification model ต้องทำได้สองอย่าง ได้แก่ การ fit ข้อมูลฝึกหัดได้ดี และการทำนาย class label ของ records ที่ไม่เคยเห็นได้อย่างแม่นยำ ซึ่งเรียกว่าความสามารถในการ generalize

กระบวนการสร้าง classification model แบ่งออกเป็นสองขั้นตอนหลัก ขั้นตอนแรกคือ Induction (Learning Phase) ซึ่งเป็นการสร้างโมเดลจาก Training Set ที่รู้ class label แล้ว Learning Algorithm จะวิเคราะห์ความสัมพันธ์ระหว่าง attributes และ class label เพื่อสร้าง classification model ขั้นตอนที่สองคือ Deduction (Prediction Phase) ซึ่งนำโมเดลที่ได้มาใช้อทำนาย class label ของ Test Set ที่ยังไม่รู้คำตอบ โมเดลที่ดีต้องมีความแม่นยำสูงทั้งบน training set และ test set การที่โมเดลแม่นยำบน training set มากเกินไปแต่แยบบน test set เรียกว่า Overfitting

6.2.2 ทฤษฎี K-Nearest Neighbors (KNN)

Manhattan Distance

$$d(x, x') = \sum_i |x_i - x'_i|$$

โดยที่:

$d(x, x')$ = ระยะทาง Manhattan ระหว่างจุด x และ x'

n = จำนวนมิติ (Dimension)

x_i = ค่าของจุด x ในมิติที่ i

x'_i = ค่าของจุด x' ในมิติที่ i

Manhattan Distance วัดระยะทางโดยรวมค่าสัมบูรณ์ของความแตกต่างในแต่ละมิติ
 เหมาะกับข้อมูลที่มีขนาด (Scale)

ต่างกันและต้องการหลีกเลี่ยงผลกระทบจากมิติเดียวที่มีค่าสูงผิดปกติ

K-Nearest Neighbors (KNN) เป็นอัลกอริทึมที่อาศัยหลักการว่า data points ที่มี feature คล้ายกันมักอยู่ใน class เดียวกัน Hastie, Tibshirani และ Friedman (2009) อธิบายว่า KNN ไม่ได้อิงสมมติฐานเชิงโครงสร้างที่เข้มงวดใดๆ เกี่ยวกับข้อมูล ทำให้สามารถปรับตัวเข้ากับสถานการณ์ได้หลากหลาย ต่างจาก Linear Models ที่ตั้งสมมติฐานว่า decision boundary เป็นเส้นตรง อย่างไรก็ตาม KNN มีแนวโน้ม high variance เนื่องจาก decision boundary ในแต่ละบริเวณขึ้นอยู่กับ data points จำนวนน้อยที่อยู่ใกล้เคียง

กระบวนการทำงานของ KNN ประกอบด้วยสามขั้นตอน ได้แก่ (1) คำนวณระยะทางระหว่าง test point กับ training points ทุกจุด โดยใช้ Euclidean Distance เป็นค่าเริ่มต้น: $d(x, x') = \sqrt{\sum_i (x_i - x'_i)^2}$ (2) เลือก k training points ที่ใกล้ที่สุด และ (3) กำหนด class ให้กับ test point ตามหลักการ majority vote จาก k neighbors ค่า k ที่เหมาะสมต้องปรับโดยการ tuning ค่า k เล็กทำให้ model มี low bias แต่ high variance ขณะที่ k ใหญ่ทำให้ decision boundary เรียบขึ้น มี low variance แต่ high bias การปรับน้ำหนักด้วย weights='distance' ทำให้ neighbors ที่ใกล้กว่ามีอิทธิพลมากกว่า ซึ่งมักให้ผลดีกว่า uniform weights ในหลายสถานการณ์

$$d(x, x') = \sqrt{\sum (x_i - x'_i)^2}$$

6.2.3 ทฤษฎี Decision Tree: การสร้างต้นไม้และเกณฑ์การแบ่ง

Gain Ratio

$$GainRatio(A) = \frac{InfoGain(A)}{SplitInfo(A)}$$

โดยที่:

GainRatio(A) = อัตราส่วนการเพิ่มข้อมูล (Gain Ratio) ของคุณลักษณะ A

InfoGain(A) = Information Gain ของคุณลักษณะ A

SplitInfo(A) = ค่าวัดการกระจายตัวของการแบ่ง (ดูสมการ SplitInfo ด้านล่าง)

Gain Ratio แก้ปัญหาของ Information Gain

ที่ลำเอียงต่อคุณลักษณะที่มีค่าหลากหลาย เช่น ID หรือ Timestamp ด้วยการหารด้วย SplitInfo

$$SplitInfo(A) = - \sum_j \frac{n_j}{n} \cdot \log \frac{n_j}{n}$$

โดยที่:

GainRatio(A) = อัตราส่วนการเพิ่มข้อมูล (Gain Ratio) ของคุณลักษณะ A

InfoGain(A) = Information Gain ของคุณลักษณะ A

SplitInfo(A) = ค่าวัดการกระจายตัวของการแบ่ง (ดูสมการ SplitInfo ด้านล่าง)

Gain Ratio แก้ปัญหาของ Information Gain

ที่ลำเอียงต่อคุณลักษณะที่มีค่าหลากหลาย เช่น ID หรือ Timestamp ด้วยการหารด้วย SplitInfo

Decision Tree สร้างขึ้นโดยอัลกอริทึม Greedy ที่แบ่ง training set ซ้ำๆ ตาม attribute ที่ให้ประโยชน์สูงสุดในแต่ละขั้น Tan et al. (2006) อธิบายว่าการแบ่งที่ดีที่สุดวัดจากความไม่บริสุทธิ์ (Impurity) ของ child nodes โดยมีเกณฑ์หลักสองประเภท

Entropy (ใช้ใน C4.5 และ ID3) คำนวณจากสมการ

$$Entropy(t) = -\sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$
 เมื่อ $p(i|t)$ คือสัดส่วนของ records ที่อยู่ใน class i ณ node t และ c คือจำนวน classes Entropy มีค่า 0 เมื่อ node บริสุทธิ์สมบูรณ์ (records ทั้งหมดเป็น class เดียวกัน) และมีค่าสูงสุด $\log_2(c)$ เมื่อการกระจาย class สม่าเสมอ อัลกอริทึมเลือก attribute ที่ลด Entropy ได้มากที่สุด ซึ่งวัดด้วย Information Gain = Entropy(parent) - $\sum (N_i / N) \times Entropy(child_i)$

$$Entropy(t) = -\sum p(i|t) \log_2 p(i|t)$$

$$InfoGain = Entropy(parent) - \sum (N_i / N) Entropy(child_i)$$

Gini Index (ใช้ใน CART) คำนวณจากสมการ $Gini(t) = 1 - \sum_{i=0}^{c-1} [p(i|t)]^2$ Gini มีค่า 0 เมื่อ node บริสุทธิ์สมบูรณ์ และมีค่า $(1 - 1/c)$ เมื่อการกระจาย class สม่าเสมอ ทั้ง Entropy และ Gini มีพฤติกรรมคล้ายกัน แต่ Entropy ลงโทษการกระจายที่ไม่สม่าเสมอ มากกว่า ในทางปฏิบัติ Gini มักคำนวณได้เร็วกว่าเพราะไม่ต้องใช้ log นอกจากนี้ยังมี Classification Error = $1 - \max_i [p(i|t)]$ แต่ใช้น้อยกว่าเพราะไม่วิธีการเปลี่ยนแปลงการกระจาย class

$$Gini(t) = 1 - \sum_{i=0}^{c-1} [p(i|t)]^2$$

ปัญหา Overfitting ใน Decision Tree เกิดจากการสร้างต้นไม้ที่ลึกเกินไปจนโมเดล “จดจำ” noise ในข้อมูลแทนที่จะเรียนรู้รูปแบบที่แท้จริง การแก้ไขทำได้โดย Pruning ซึ่งมีสองแนวทาง ได้แก่ Pre-pruning (Early Stopping) ที่หยุดสร้างต้นไม้ก่อนผ่านการกำหนด max_depth หรือ min_samples_split และ Post-pruning ที่สร้างต้นไม้เต็มก่อนแล้วค่อยตัดกิ่งที่ไม่จำเป็นออก

6.2.4 ทฤษฎี Support Vector Machine (SVM)

SVM ทำงานโดยค้นหา Hyperplane ที่แบ่ง data points ของสองคลาสออกจากกัน โดยเพิ่ม Margin ระหว่าง Hyperplane กับ data points ที่ใกล้ที่สุดของแต่ละคลาส (เรียกว่า

Support Vectors) ให้มากที่สุด หลักการ Maximum Margin มีพื้นฐานทางทฤษฎีรองรับว่าช่วยลด generalization error ได้ดี Hyperplane ที่เหมาะสมนิยามด้วยสมการ

$$w^T x + b = 0 \text{ โดย } w \text{ คือ weight vector และ } b \text{ คือ bias ค่า } \text{Margin} = \frac{2}{\|w\|} \text{ ดังนั้น}$$

การเพิ่ม Margin สูงสุดเทียบเท่ากับการลด $\|w\|^2$ ภายใต้ constraints ที่กำหนด

$$w^T x + b = 0$$

$$\text{Margin} = \frac{2}{\|w\|}$$

ค่า Regularization Parameter C ควบคุมความสมดุลระหว่าง Margin กับการยอมรับข้อผิดพลาด ค่า C สูงทำให้โมเดลพยายามจำแนกทุก training point ให้ถูกต้อง (Hard Margin) ส่งผลให้ Margin แคบลงและเสี่ยง Overfitting ค่า C ต่ำยอมให้มีการจำแนกผิดพลาดบาง training points (Soft Margin) ส่งผลให้ Margin กว้างขึ้นและโมเดล generalize ได้ดีขึ้น สำหรับข้อมูลที่ไม่สามารถแบ่งแยกแบบเส้นตรงได้ (Non-linearly Separable) SVM ใช้ Kernel Trick ซึ่งแปลงข้อมูลไปยัง high-dimensional feature space โดยไม่ต้องคำนวณ transformation ตรงๆ Kernel ที่นิยมใช้ได้แก่ Linear Kernel, RBF (Radial Basis Function) Kernel และ Polynomial Kernel

6.2.5 ทฤษฎี Naive Bayes Classifier

Naive Bayes อิงจากทฤษฎีบท Bayes โดยคำนวณความน่าจะเป็นที่ record x จะอยู่ใน class C_r เป็น และเลือก class ที่ให้ค่า posterior probability สูงสุด ข้อสมมติฐาน “Naive” คือการสมมติว่า attributes ทุกตัวมีความเป็นอิสระต่อกันแบบมีเงื่อนไข (Conditionally Independent) เมื่อรู้ class ซึ่งทำให้ $p(x|c_r) = \prod_k p(x^k|c_r)$ สมมติฐานนี้แม้จะไม่เป็นจริงในทางปฏิบัติ แต่ก็ทำให้การประมาณค่าพารามิเตอร์ง่ายขึ้นอย่างมาก เพราะต้องประมาณค่า $P(x^k|C_r)$ แยกกันทีละ attribute แทนที่จะต้องประมาณ joint distribution ทั้งหมด

$$P(C_r | x) \propto P(x | C_r) \times P(C_r)$$

Hastie et al. (2009) อธิบายว่า แม้ Naive Bayes จะมีสมมติฐานที่ดูเรียบง่ายเกินไป แต่กลับมักให้ผลดีมากกว่า alternatives ที่ซับซ้อนกว่าในหลายกรณี เหตุผลเบื้องหลังคือ แม้ class-conditional densities แต่ละตัวจะมี bias แต่ bias เหล่านั้นอาจไม่กระทบต่อ posterior probabilities ใกล้เคียงกับ decision boundary มากนัก นอกจากนี้สมมติฐาน conditional independence ช่วยลด variance ของโมเดลได้มาก ทำให้ trade-off ระหว่าง bias-variance เอื้อต่อ Naive Bayes โดยเฉพาะเมื่อข้อมูลมีจำนวนน้อยหรือ feature มีจำนวนมาก Gaussian Naive Bayes สมมติให้ $P(x^k|C_r)$ มีการกระจายแบบ Normal Distribution

6.2.6 ทฤษฎี Logistic Regression

Softmax Function

$$P(y = k | x) = \frac{e^{w_k \cdot x}}{\sum_i e^{w_i \cdot x}}$$

โดยที่:

$P(y=k|x)$ = ความน่าจะเป็นที่ข้อมูล x จัดอยู่ในคลาส k

z_k = คะแนนดิบ (Logit) ของคลาส k ก่อนผ่าน Softmax

K = จำนวนคลาสทั้งหมด

ฟังก์ชัน Softmax แปลงคะแนนดิบให้กลายเป็นความน่าจะเป็นที่รวมกันได้ 1 ใช้เป็น

Output Layer ของโครงข่ายประสาทเทียมสำหรับการจำแนกหลายคลาส

Logistic Regression แม้จะมีชื่อว่า Regression แต่ใช้สำหรับงาน Classification โดยสร้างโมเดลความน่าจะเป็นแบบ log-odds ผ่าน logit function:

$\log(P(Y=1|x)/P(Y=0|x)) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$ หรือเทียบเท่ากับ

$P(Y=1|x) = 1 / (1 + e^{-\beta^T x})$ ซึ่งเรียกว่า Sigmoid Function ผลลัพธ์คือค่าความน่าจะเป็นในช่วง (0, 1) และจำแนก class ตาม threshold (ปกติ 0.5) Logistic Regression มีรากฐานทางทฤษฎีที่แข็งแกร่งจากสถิติคลาสสิก สามารถตีความ coefficients ได้ตรงไปตรงมา และมีประสิทธิภาพสูงเมื่อ decision boundary เป็นเส้นตรง

$$P(Y = 1 | x) = \frac{1}{1 + e^{-\beta^T x}}$$

Regularization parameter C ใน Logistic Regression ของ Scikit-learn มีความหมายตรงข้ามกับ SVM ในบริบทนี้ $C = 1/\lambda$ เมื่อ λ คือ regularization strength ดังนั้น C ต่ำ = regularization สูง = โมเดลเรียบง่าย และ C สูง = regularization ต่ำ = โมเดลซับซ้อน Regularization ช่วยป้องกัน Overfitting โดยลงโทษ coefficients ที่มีขนาดใหญ่ผ่าน L1 (Lasso) หรือ L2 (Ridge) penalty

6.2.7 การประเมินประสิทธิภาพของโมเดลจำแนกประเภท

F β Score

$$F_\beta = \frac{(1 + \beta) \cdot P \cdot R}{\beta \cdot P + R}$$

โดยที่:

F_β = ค่า F-score ถ่วงน้ำหนักด้วยพารามิเตอร์ β

P = Precision = TP / (TP + FP)

R = Recall = TP / (TP + FN)

β = พารามิเตอร์ควบคุมน้ำหนัก: $\beta < 1$ ให้น้ำหนัก Precision มากกว่า, $\beta > 1$ ให้น้ำหนัก Recall มากกว่า

กรณีพิเศษ: F_1 Score ($\beta=1$) ถ่วงน้ำหนัก Precision และ Recall เท่ากัน, F_2 Score ($\beta=2$) เน้น Recall สองเท่าของ Precision

Tan et al. (2006) นำเสนอการวัดประสิทธิภาพที่สำคัญจาก Confusion Matrix ซึ่งมีองค์ประกอบหลักคือ TP (True Positive), TN (True Negative), FP (False Positive) และ FN (False Negative) จากนั้นสามารถคำนวณได้ดังนี้ Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ วัดสัดส่วนการทำนายที่ถูกต้อง

Precision = $\frac{TP}{TP + FP}$ ทั้งหมด Precision = $TP / (TP + FP)$ วัดความแม่นยำในการ

ระบุ positive cases $Recall (Sensitivity) = TP / (TP + FN)$ วัดความครบถ้วนในการค้นหาค่า positive cases ที่ 'มี' อยู่จริง และ $F1-Score = 2 \times (Precision \times Recall) / (Precision + Recall)$. เป็นค่า harmonic mean ระหว่าง Precision และ Recall เหมาะสำหรับข้อมูล imbalanced

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

ตารางที่ 6.1 สรุปเปรียบเทียบลักษณะสำคัญของอัลกอริทึม Classification หลักในบทนี้

อัลกอริทึม	หลักการหลัก	จุดแข็ง	ข้อจำกัด
KNN	Majority vote จาก k neighbors ที่ใกล้ที่สุด	ไม่ต้องสมมติฐาน, ปรับตัวได้	ช้าเมื่อข้อมูลใหญ่, ไวต่อ scaling
Decision Tree	แบ่งข้อมูลซ้ำตาม Gini/Entropy	ตีความง่าย, รองรับข้อมูลหลายประเภท	Overfitting ง่าย, ต้องการ Pruning
SVM	Maximum Margin Hyperplane	รองรับ high-dim, Kernel trick	ช้า, ต้องเลือก Kernel และ C ที่เหมาะสม
Naive Bayes	Bayes Theorem + Conditional Independence	เร็ว, ดีกับข้อมูลน้อย/feature เยอะ	สมมติ independence อาจไม่จริง
Logistic Regression	Sigmoid Function โมเดลความน่าจะเป็น	ตีความได้, มีรากฐานทางสถิติ	จำกัดกับ linear decision boundary

ที่มา: ปรับปรุงจาก Tan, Steinbach, and Kumar (2006) และ Hastie, Tibshirani, and Friedman (2009)

6.3 ตัวจำแนกประเภทแบบ Nearest Neighbor

ตัวจำแนกประเภทแบบ Nearest Neighbor เป็นอัลกอริทึมพื้นฐานที่ใช้ความใกล้ชิดของจุดข้อมูลในการทำนาย K-Nearest Neighbors (KNN) เป็นอัลกอริทึม Supervised Machine Learning ที่ใช้สำหรับงาน Classification และ Regression โดยอาศัยหลักการที่ว่าจุดข้อมูลที่มี Feature คล้ายกันมักจะอยู่ในคลาสเดียวกัน Radius Neighbors (RNN) เป็นส่วนขยายของ KNN ที่เน้นจุดข้อมูลภายในรัศมีหรือระยะทางที่กำหนดจากจุดที่ต้องการสืบค้น

6.3.1 แบบฝึกหัด – การจำแนก Iris แบบ Binary ด้วย KNN

เ อ ก ส า ร อ ั ว ง อ ิ ง : <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

6.3.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
iris = datasets.load_iris()
df = pd.DataFrame({'Sepal length':
iris.data[:,0], 'Sepal width': iris.data[:,1],
'Petal length': iris.data[:,2], 'Petal width':
iris.data[:,3],
'Species': iris.target})
df = df[df['Species'] != 0] # เลือกเฉพาะ 2 คลาส (Binary)
```

6.3.1.2 แบ่งข้อมูล Train-Test และ Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X = df[df.columns[:2]]
y = df[df.columns[-1]]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.50)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

6.3.1.3 สร้างและประเมินโมเดล KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
classifier = KNeighborsClassifier(n_neighbors=1)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test, y_pred)) # 0.54
```

6.3.1.4 หาค่า k ที่ดีที่สุด

```
def knn_tuning(k):
    classifier = KNeighborsClassifier(n_neighbors=k)
    classifier.fit(X_train, y_train)
```

```

        return accuracy_score(y_test,
                               classifier.predict(X_test))

knn_results = pd.DataFrame({'K': np.arange(1,
len(X_train), 5)})

knn_results['Accuracy'] =
knn_results['K'].apply(knn_tuning)

```

ผลลัพธ์: k=11 และ k=16 ให้ความแม่นยำสูงสุดที่ 0.66

6.3.1.5 ปรับแต่ง Weights

```

knn_results['Uniform'] =
knn_results['K'].apply(lambda k:
KNeighborsClassifier(n_neighbors=k,
weights='uniform').fit(X_train,y_train).score(X
_test,y_test))

knn_results['Distance'] =
knn_results['K'].apply(lambda k:
KNeighborsClassifier(n_neighbors=k,
weights='distance').fit(X_train,y_train).score(
X_test,y_test))

```

6.3.2 แบบฝึกหัด – การจำแนก Iris แบบ Multiclass ด้วย KNN

ใช้ชุดข้อมูล Iris ทั้ง 3 คลาส โดยแบ่ง Test 20% และใช้ 4 Feature ทั้งหมด

```

X = df[df.columns[:4]]

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.20)

```

ผลลัพธ์: k=6 และ k=11 ให้ความแม่นยำสูงสุดที่ 1.00 (100%) การใช้ weights='distance' ยังคงรักษาความแม่นยำสูงได้แม้ k จะมากขึ้น

6.3.3 แบบฝึกหัด – การจำแนก Iris แบบ Binary ด้วย RNN

```

from sklearn.neighbors import
RadiusNeighborsClassifier

classifier =
RadiusNeighborsClassifier(radius=1)

```

```
classifier.fit(X_train, y_train)
print('Accuracy:', accuracy_score(y_test,
classifier.predict(X_test))) # 0.60
```

ผลลัพธ์การทดสอบค่า r ต่างๆ: $r=1.0$ และ $r=2.5$ ให้ความแม่นยำสูงสุดที่ 0.60 ค่า r ที่สูงขึ้นทำให้ความแม่นยำลดลง

6.3.4 แบบฝึกหัด – การจำแนก Iris แบบ Multiclass ด้วย RNN

ใช้ Iris ทั้ง 3 คลาส ด้วย $r=1.0$ ให้ความแม่นยำ 100% แต่เมื่อ r เพิ่มขึ้น ความแม่นยำจะลดลงอย่างรวดเร็ว การใช้ `weights='distance'` ช่วยรักษาความแม่นยำสูงได้ดีกว่า `weights='uniform'` เมื่อ r มีค่าสูง

6.3.5 กรณีศึกษา – การจำแนกมะเร็งเต้านมด้วย Nearest Neighbor

ใช้ชุดข้อมูล Breast Cancer จาก Scikit-learn ประกอบด้วย Feature 30 รายการ เพื่อจำแนกว่าเนื้องอกเป็นมะเร็ง (malignant) หรือไม่ (benign) แบ่งข้อมูล 80% สำหรับฝึก และ 20% สำหรับทดสอบ

6.3.5.1 การตั้งค่า

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X = pd.DataFrame(data.data,
columns=data.feature_names)
y = pd.Series(data.target)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)
```

6.3.5.2 ทดสอบ KNN ด้วย k ต่างๆ

```
k_values = [1, 5, 11, 15, 21]
for k in k_values:
```

```

knn_model =
KNeighborsClassifier(n_neighbors=k)
knn_model.fit(X_train, y_train)
print(f'K={k}:
Accuracy={accuracy_score(y_test,
knn_model.predict(X_test)):.2f}')

```

ผลลัพธ์ KNN: K=1: 0.93, K=5: 0.96, K=11: 0.98, K=15: 0.96, K=21: 0.96 — K=11 ให้ผลดีที่สุด

6.3.5.3 ทดสอบ RNN ด้วย Radius ต่างๆ

```
radius_values = [350, 400, 450, 500, 550, 600]
```

ผลลัพธ์ RNN: Radius=350: 0.94, Radius=400: 0.94, Radius=450: 0.94, Radius=500: 0.91, Radius=550: 0.90, Radius=600: 0.90 — KNN (K=11) ให้ผลดีกว่า RNN

6.4 ตัวจำแนกประเภทแบบ Decision Tree

Decision Tree เป็นอัลกอริทึม Machine Learning ที่ทรงพลังและนิยมใช้สำหรับงาน Classification เนื่องจากความสามารถในการตีความและความเรียบง่าย ทำงานโดยการแบ่งชุดข้อมูลซ้ำๆ ตาม Feature ที่มีข้อมูลเชิงลึกมากที่สุดเพื่อทำนาย สามารถปรับแต่งได้ด้วยเกณฑ์การแบ่ง (criterion) และการตัดแต่ง (pruning) เพื่อป้องกัน Overfitting

6.4.1 แบบฝึกหัด – การจำแนก Iris แบบ Binary ด้วย Decision Tree

เอกสารอ้างอิง : <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

6.4.1.1 การตั้งค่าและโหลดข้อมูล

```

from sklearn.tree import DecisionTreeClassifier
iris = datasets.load_iris()
df = pd.DataFrame({'Sepal length':
iris.data[:,0], 'Sepal width': iris.data[:,1],

```

```
'Petal length': iris.data[:,2], 'Petal width':
iris.data[:,3], 'Species': iris.target})
df = df[df['Species'] != 0]
X_train, X_test, y_train, y_test =
train_test_split(df[df.columns[:4]],
df[df.columns[-1]], test_size=0.20)
```

6.4.1.2 สร้างและประเมินโมเดล

```
classifier = DecisionTreeClassifier()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test,
y_pred)) # 0.95
```

6.4.1.3 แสดงผลต้นไม้ Decision

```
from sklearn import tree
print(tree.export_text(classifier))
fig = plt.figure(figsize=(10,8))
tree.plot_tree(classifier,
feature_names=iris.feature_names,
class_names=iris.target_names, filled=True)
```

6.4.1.4 ปรับแต่ง Hyperparameters

ทดลองปรับ criterion และ max_depth:

```
classifier =
DecisionTreeClassifier(criterion='entropy') # ใช้
Entropy
classifier =
DecisionTreeClassifier(max_depth=1) # Accuracy:
0.95
```



```

classifier =
DecisionTreeClassifier(max_depth=2) # Accuracy:
1.00 (ดีที่สุด)

classifier =
DecisionTreeClassifier(max_depth=3) # Accuracy:
0.95

def tree_depth_tuning(d):
    clf =
DecisionTreeClassifier(max_depth=d).fit(X_train
, y_train)
    return accuracy_score(y_test,
clf.predict(X_test))

tree_results = pd.DataFrame({'D':
np.arange(1,10)})

tree_results['Accuracy'] =
tree_results['D'].apply(tree_depth_tuning)

```

ผลลัพธ์: max_depth=2 ให้ความแม่นยำสูงสุดที่ 1.00 (100%)

6.4.2 แบบฝึกหัด – การจำแนก Iris แบบ Multiclass ด้วย Decision Tree

ใช้ Iris ทั้ง 3 คลาสพร้อม 4 Feature แบ่ง Test 20% ผลลัพธ์เริ่มต้น Accuracy: 0.933
เมื่อปรับ max_depth=2 ได้ Accuracy: 0.967 และ max_depth=3 ได้ Accuracy: 0.967
เช่นกัน

6.4.3 กรณีศึกษา – การจำแนกมะเร็งเต้านมด้วย Decision Tree

ทดสอบ criterion 2 แบบ (entropy, gini) ร่วมกับ max_depth หลายค่า (1, 3, 5, ..., 19)

6.4.3.1 การตั้งค่า

```

splitting_criteria = ['entropy', 'gini']
max_depth_values = range(1, 20, 2)
for criterion in splitting_criteria:

```

```

for max_depth in max_depth_values:
    dt_model = DecisionTreeClassifier(criterion=criterion,
max_depth=max_depth)
    dt_model.fit(X_train, y_train)
    accuracy = accuracy_score(y_test,
dt_model.predict(X_test))

```

6.4.3.2 ผลลัพธ์

Entropy: max_depth=3,5,13,17 ให้ ความแม่นยำ สูงสุด ที่ 0.96 / Gini: max_depth=3,11,15 ให้ความแม่นยำสูงสุดที่ 0.95 / โดยรวม Entropy ให้ผลดีกว่า Gini เล็กน้อย

6.5 ตัวจำแนกประเภทแบบ Support Vector Machine (SVM)

Support Vector Machines (SVMs) เป็นอัลกอริทึม Machine Learning ที่ทรงพลังและหลากหลาย ทำงานโดยการค้นหา Hyperplane ที่เหมาะสมที่สุดเพื่อแยกข้อมูลออกเป็นคลาสต่างๆ Scikit-learn ใช้ คลาส SVC (Support Vector Classification) พร้อมพารามิเตอร์สำคัญ ได้แก่ ประเภท Kernel (linear, rbf, poly), ค่า C (Regularization) และ gamma

6.5.1 แบบฝึกหัด – การจำแนก Iris แบบ Binary ด้วย SVM

เอกสารอ้างอิง: <https://scikit-learn.org/stable/modules/svm.html>

```

from sklearn import svm
model = svm.SVC(kernel='linear')
classifier = model.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
print('Accuracy:', accuracy_score(y_test,
y_pred)) # 0.90

```

6.5.2 แบบฝึกหัด – การจำแนก Iris แบบ Multiclass ด้วย SVM

ใช้ 4 Feature ทั้งหมด แบ่ง Test 20% ผลลัพธ์: Accuracy = 0.967 — SVM สามารถแยก 3 คลาสได้ดีมาก

6.5.3 กรณีศึกษา – การจำแนกมะเร็งเต้านมด้วย SVM

ทดสอบ Regularization Parameter C หลายค่า: [0.01, 0.1, 1, 10, 100]

```
from sklearn.svm import SVC
C_values = [0.01, 0.1, 1, 10, 100]
for C in C_values:
    svm_model = SVC(C=C, kernel='linear')
    svm_model.fit(X_train, y_train)
    print(f'C={C}:
    Accuracy={accuracy_score(y_test,
    svm_model.predict(X_test)):.4f}')
```

ผลลัพธ์: C=0.01: 0.9561, C=0.1: 0.9649 (ดีที่สุด), C=1: 0.9561, C=10: 0.9561, C=100: 0.9561 — C ที่ต่ำเกินไปหรือสูงเกินไปล้วนลด Accuracy

6.6 ตัวจำแนกประเภทแบบ Naive Bayes

Naive Bayes เป็นอัลกอริทึม Machine Learning เชิงความน่าจะเป็นที่ใช้กันทั่วไปสำหรับงาน Classification โดยเฉพาะในการประมวลผลภาษาธรรมชาติและวิเคราะห์ข้อความอิงจากทฤษฎีของ Bayes และสมมติว่า Feature มีอิสระต่อกัน (ดังนั้นจึงชื่อว่า 'Naive') Scikit-learn รองรับหลายประเภท ได้แก่ GaussianNB, MultinomialNB และ BernoulliNB

6.6.1 แบบฝึกหัด – การจำแนก Iris แบบ Binary ด้วย Naive Bayes

เอกสารอ้างอิง: https://scikit-learn.org/stable/modules/naive_bayes.html

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
print('Accuracy:', accuracy_score(y_test,
y_pred)) # 0.85
```

6.6.2 แบบฝึกหัด – การจำแนก Iris แบบ Multiclass ด้วย Naive Bayes

ใช้ 4 Feature ทั้งหมด แบ่ง Test 20% ผลลัพธ์: Accuracy = 0.933 — Naive Bayes ทำงานได้ดีแม้จะมีสมมติฐานที่เรียบง่าย

6.6.3 กรณีศึกษา – การจำแนกมะเร็งเต้านมด้วย Naive Bayes

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
accuracy = accuracy_score(y_test,
nb_model.predict(X_test))
print('Accuracy:', accuracy) # 0.9737
```

ผลลัพธ์: Naive Bayes ให้ความแม่นยำ 97.37% สำหรับชุดข้อมูลมะเร็งเต้านม ซึ่งเป็นผลที่ดีเยี่ยมแม้จะเป็นอัลกอริทึมที่เรียบง่าย

6.7 ตัวจำแนกประเภทแบบ Logistic Regression

Logistic Regression เป็นเทคนิคสถิติและ Machine Learning ที่นิยมใช้สำหรับงาน Binary Classification โมเดลความน่าจะเป็นของผลลัพธ์แบบ Binary ตามตัวแปรทำนายหนึ่งตัวหรือมากกว่า แม้จะมีชื่อว่า 'Regression' แต่ใช้สำหรับการจำแนกประเภท Scikit-learn ใช้คลาส LogisticRegression พร้อมพารามิเตอร์ C สำหรับ Regularization

6.7.1 แบบฝึกหัด – การจำแนก Iris แบบ Binary ด้วย Logistic Regression

เอกสารอ้างอิง : https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

```
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred)) # 0.75
```

6.7.2 แบบฝึกหัด – การจำแนก Iris แบบ Multiclass ด้วย Logistic Regression

ใช้ 4 Feature ทั้งหมด แบ่ง Test 20% ผลลัพธ์: Accuracy = 0.933

6.7.3 กรณีศึกษา – การจำแนกมะเร็งเต้านมด้วย Logistic Regression

ทดสอบค่า C หลายค่า: [0.001, 0.01, 0.1, 1, 10, 100, 1000]

```
C_values = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
for C in C_values:
    lr_model = LogisticRegression(C=C,
    random_state=42, max_iter=2000)
    lr_model.fit(X_train, y_train)
    print(f'C={C}:
    Accuracy={accuracy_score(y_test,
    lr_model.predict(X_test)):.4f}')
```

ผลลัพธ์: C=0.001: 0.9649, C=0.01: 0.9649, C=0.1: 0.9649, C=1: 0.9561, C=10: 0.9561, C=100: 0.9649, C=1000: 0.9649 — C ที่ต่ำ (Regularization สูง) ให้ผลดีกว่าสำหรับชุดข้อมูลนี้

6.8 การเปรียบเทียบวิธีการจำแนกประเภท

ส่วนนี้นำเสนอการศึกษาเชิงเปรียบเทียบที่ครอบคลุมสำหรับวิธีการจำแนกประเภทต่างๆ โดยใช้ชุดข้อมูลเดียวกัน เพื่อให้เข้าใจถึงจุดแข็งและจุดอ่อนของแต่ละอัลกอริทึม และสามารถเลือกอัลกอริทึมที่เหมาะสมที่สุดสำหรับงานที่กำหนด

6.8.1 กรณีศึกษา – การจำแนกไวน์ด้วยหลายอัลกอริทึม

ใช้ Wine Dataset ซึ่งเป็นชุดข้อมูลยอดนิยมสำหรับงาน Classification โดยคลาสเป้าหมายแทนแหล่งกำเนิดของไวน์ต่างๆ มี 13 Feature ที่อธิบายคุณลักษณะต่างๆ ของไวน์

6.8.1.1 การตั้งค่า

```
from sklearn.datasets import load_wine
data = load_wine()
X = pd.DataFrame(data.data,
                  columns=data.feature_names)
y = pd.Series(data.target, name='target')
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
                  random_state=42)
```

6.8.1.2 ผลการเปรียบเทียบ

KNN (k=37-49): Accuracy สูงสุด 0.81 / Decision Tree (max_depth=3-19): Accuracy สูงสุด 0.94 / SVM (C=100-200): Accuracy สูงสุด 0.83 / Naive Bayes: Accuracy 1.00 (ดีที่สุด!) / Logistic Regression (C=0.01-0.1): Accuracy สูงสุด 1.00

สรุป: สำหรับ Wine Dataset Naive Bayes และ Logistic Regression ให้ความแม่นยำสูงสุดที่ 100% ในขณะที่ Decision Tree ให้ผล 94% และ KNN กับ SVM ให้ผลต่ำกว่า

6.8.1.3 แสดงผลกราฟเปรียบเทียบ

```
plt.figure(figsize=(12, 6))
plt.subplot(2, 2, 1)
```

```
plt.plot(list(knn_results.keys()),
list(knn_results.values()), marker='o')
plt.title('Accuracy vs k for KNN')
plt.subplot(2, 2, 2)
plt.plot(list(dt_results.keys()),
list(dt_results.values()), marker='o')
plt.title('Accuracy vs Max Depth for Decision
Trees')
plt.subplot(2, 2, 3)
plt.plot(C_values, list(svm_results.values()),
marker='o')
plt.title('Accuracy vs C for SVM')
plt.subplot(2, 2, 4)
plt.plot(C_values,
list(logreg_results.values()), marker='o')
plt.title('Accuracy vs C for Logistic
Regression')
plt.tight_layout()
plt.show()
```

ตัวอย่างการประยุกต์ใช้งานจากงานวิจัย

สิมมาจันท์และคณะ (2023) ได้ประยุกต์ใช้การจำแนกประเภทข้อมูลในงานวิจัยเรื่อง "Detecting Fraudulent Claims in Automobile Insurance Policies by Data Mining Techniques" โดยเปรียบเทียบอัลกอริทึม 4 ตัว ได้แก่ Logistic Regression, Random Forest, Naive Bayes และ Adaptive Boosting (AdaBoost) สำหรับการตรวจจับการฉ้อโกงเรียกร้องค่าสินไหมประกันภัยรถยนต์ ผลการศึกษาพบว่า Random Forest ให้ผลลัพธ์ที่ดีที่สุด โดยมีค่า Sensitivity สูงถึง 99.19% และ Positive Predictive Value 93.62% สะท้อนให้เห็น ประสิทธิภาพของ Non-parametric Classifier ในงานข้อมูลที่ไม่สมดุล

ผาพันธ์และคณะ (2025) ได้พัฒนาต่อยอดด้วยกรอบแนวคิด Two-step Models ในงานวิจัยเรื่อง "Detecting Automobile Insurance Fraud: A Novel Two-Step Strategy Using Effective Ensemble Learning Techniques" โดยในขั้นแรก สร้างแบบจำลองพื้นฐาน 8 ตัวแล้วคัดเลือก 3 ตัวที่ดีที่สุดตามคะแนน F-measure จากนั้นในขั้นที่สองนำผลการทำนายของทั้งสามมารวมกันด้วยเทคนิค Ensemble พบว่าการรวม SMOTE กับ Improved Multilayer Perceptron (IMLP) ให้ประสิทธิภาพสูงสุด

วงศ์วิวัฒน์และคณะ (2025) ในงานวิจัยเรื่อง "Machine Learning Approaches for Credit Risk Assessment in SMEs" ได้นำอัลกอริทึม Decision Tree, Support Vector Machine (SVM), Logistic Regression และ AdaBoost มาเปรียบเทียบสำหรับการประเมินความเสี่ยงสินเชื่อของวิสาหกิจขนาดกลาง และขนาดย่อม (SMEs) โดยประเมินด้วย K-fold Cross-validation ผลพบว่า AdaBoost ให้ความแม่นยำโดยรวมสูงสุดหลังจากการทำ Data Balancing ซึ่งสอดคล้องกับแนวคิดของ Ensemble Learning ที่กล่าวถึงในบทนี้

เอกสารอ้างอิง

1. Simmachan, T., Manopa, W., Neamhom, P., Poothong, A., & Phaphan, W. (2023). Detecting fraudulent claims in automobile insurance policies by data mining techniques. *Thailand Statistician*, 21(3), 552–568.
2. Phaphan, W., Sathitvudh, S., Suntornsuwan, T., Budsaba, K., & Simmachan, T. (2025). Detecting automobile insurance fraud: A novel two-step strategy using effective ensemble learning techniques. *Thailand Statistician*, 23(1), 144–161.

3. Wongwiwat, P., Chairojwattana, A., & Phaphan, W. (2025). Machine learning approaches for credit risk assessment in SMEs. *Thailand Statistician*, 23(3), 710–731.